

módulo auth

Documentación técnica completa — Spring Boot + Hexagonal + JWT

Spring Security

JWT / HMAC-SHA256

Hexagonal Architecture

Spring Modulith

1. Arquitectura General

El módulo sigue arquitectura **hexagonal estricta**. El controlador REST habla con **puertos de entrada**, los servicios de aplicación hablan con **puertos de salida**, y la infraestructura implementa esos puertos. Spring Security vive completamente en infraestructura, invisible para el dominio.

Capa	Clases principales	Responsabilidad
API	AuthController	Expone /api/auth/**
Application	RegisterUserService AuthenticateUserService	Orquesta casos de uso
Domain	User · Role · UserRegisteredEvent	Entidades y eventos
Infraestructura	JwtService · JwtAuthFilter UserDetailsAdapter · UserPersistenceAdapter	JWT, Spring Security, JPA

2. Flujo — Registro POST /api/auth/register

POST /api/auth/register

AuthController

→ RegisterUserService.register(command)

1. loadUserPort.loadByEmail() // email único

si existe → EmailAlreadyTakenException (409)

2. User.create(...) // UUID propio + isNew=true

3. passwordEncoder.encode() // BCrypt

4. saveUserPort.save(user) // persiste en DB

5. eventPublisher.publish(UserRegisteredEvent)

6. tokenGeneratorPort.generate() // genera JWT

← AuthResult { token, userId, email, role }

Detalle crítico — Persistable<UUID>: La entidad User asigna su propio UUID (no la DB). Por eso implementa Persistable<UUID> con isNew = true al crear y isNew = false en @PostLoad. Sin esto, JPA hace un SELECT antes del INSERT — lo que causaba el bug 403 documentado en docs/bugfix-auth-403-register.md.

3. Flujo — Login POST /api/auth/login

[illegible]

```
→ AuthenticateUserService.authenticate(command)
```

→ `InvalidCredentialsException` (401)

→ si falla → `InvalidCredentialsException` (401)

```
← AuthResult { token, userId, email, role }
```

InvalidCredentialsException. Nunca se le dice al cliente cuál de los dos falló.

4. Validación JWT en cada request

Cada request autenticado pasa por `JwtAuthFilter` antes de llegar al controlador. Extiende `OncePerRequestFilter` — se ejecuta exactamente una vez por request.

```
1. Lee header "Authorization"
```

```
3. jwtService.validateAndExtract(token)
```

4. Extrae email del claim

```

    UserDetailsAdapter → LoadUserPort.loadByEmail()

```

6. Injecta UsernamePasswordAu

```
en SecurityContextHolder
```

```
// Spring Security rechazará con 401 si el endpoint lo requiere
```

5. Configuración de Seguridad — SecurityConfig

Ruta	Acceso	Motivo
/api/auth/**	Público	Registro y login
/actuator/health	Público	Healthcheck
/error	Público	Página de error de Spring
/api/stores/plans	Público	Ver planes sin estar logueado
/api/stores/{slug}	Público	Ver tienda pública
/api/billing/webhook/**	Público	Webhooks de pasarelas de pago
anyRequest()	Autenticado	Todo lo demás requiere JWT

```
// Sin sesión, sin CSRF, @EnableMethodSecurity para @PreAuthorize
sessionManagement(sm → sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
csrf(AbstractHttpConfigurer::disable)
```

6. Generación y Claims del JWT — JwtService

```
// Claims que se incluyen en cada token
sub → user.getId().toString() // UUID
email → user.getEmail()
role → user.getRole().name() // "SELLER" o "BUYER"
iat → timestamp de emisión
exp → iat + expirationMs

// Firmado con HMAC-SHA256 usando la clave del secret
```

Configuración en application.yml:

```
jwt:
secret: ${JWT_SECRET:livecommerce-secret-key-change-this-in-production-min-32chars}
expiration-ms: ${JWT_EXPIRATION_MS:86400000} # 24 horas
```

El secret se inyecta vía JwtProperties — un record con @ConfigurationProperties(prefix = "jwt"). El default tiene 48 chars — suficiente para HMAC-SHA256 que requiere mínimo 32. En producción sobrescribís con la variable de entorno JWT_SECRET.

7. UserPrincipal — objeto en el SecurityContext

Vive en shared/ porque otros módulos (order, store) lo usan para obtener el userId del usuario autenticado sin depender del módulo auth. Implementa UserDetails de Spring pero agrega getUserId().

```
// En cualquier handler autenticado
var principal = (UserPrincipal) SecurityContextHolder.getContext()
    .getAuthentication().getPrincipal();
UUID userId = principal.getUserId();
```

8. Mapa completo de dependencias

```
[domain]
User ■■ Persistable<UUID> (Spring Data)
Role (enum: SELLER, BUYER)
UserRegisteredEvent

[application ports]
in: RegisterUserUseCase, AuthenticateUserUseCase
out: LoadUserPort, SaveUserPort, TokenGeneratorPort

[application services]
RegisterUserService → implements RegisterUserUseCase
AuthenticateUserService → implements AuthenticateUserUseCase
```

```
[infrastructure]
JwtService → implements TokenGeneratorPort
UserPersistenceAdapter → implements LoadUserPort + SaveUserPort
UserJpaRepository (Spring Data JPA)
RoleConverter (enum ↔ String en DB)
UserDetailsAdapter → implements UserDetailsService
JwtAuthFilter → OncePerRequestFilter
SecurityConfig → wires Spring Security
JwtProperties → @ConfigurationProperties("jwt")
```

Generado con graphify knowledge graph · livecommerce project